Assignment 06 Visitor Pattern

Koishybay Zhasulan | SE-2537

Purpose and design

The program models files and directories in memory. Three visitors use the same tree to calculate size, find files, and count files, directories, and depth.
Double dispatch has two steps. Java first calls accept on the actual node. Inside accept, the node passes this to the matching visit overload. Java then runs that overload on the actual visitor.

Traversal and extension

TreeWalker alone performs a recursive depth-first walk. Each node is visited once. The root is depth 0 and counts if it is a directory. Paths start below the root, use /, and sort lexicographically. A matching file root has path ".". Visitors reset before each run.
StatisticsVisitor uses the existing visit methods; the nodes and accept methods do not change. Adding a new element type costs more: the sealed hierarchy, Visitor interface, and existing visitors would need updates.

Five Clean Code practices

Meaningful names: minimumSize makes the search limit clear.
Single responsibility: TotalSizeVisitor adds file sizes; TreeWalker handles traversal.
Small methods: FileNode.accept only forwards the call.
Encapsulation: DirectoryNode.children returns an immutable snapshot; add checks ownership, names, and cycles.
No type checks: each node calls its own visit overload, so the walker never inspects node types.

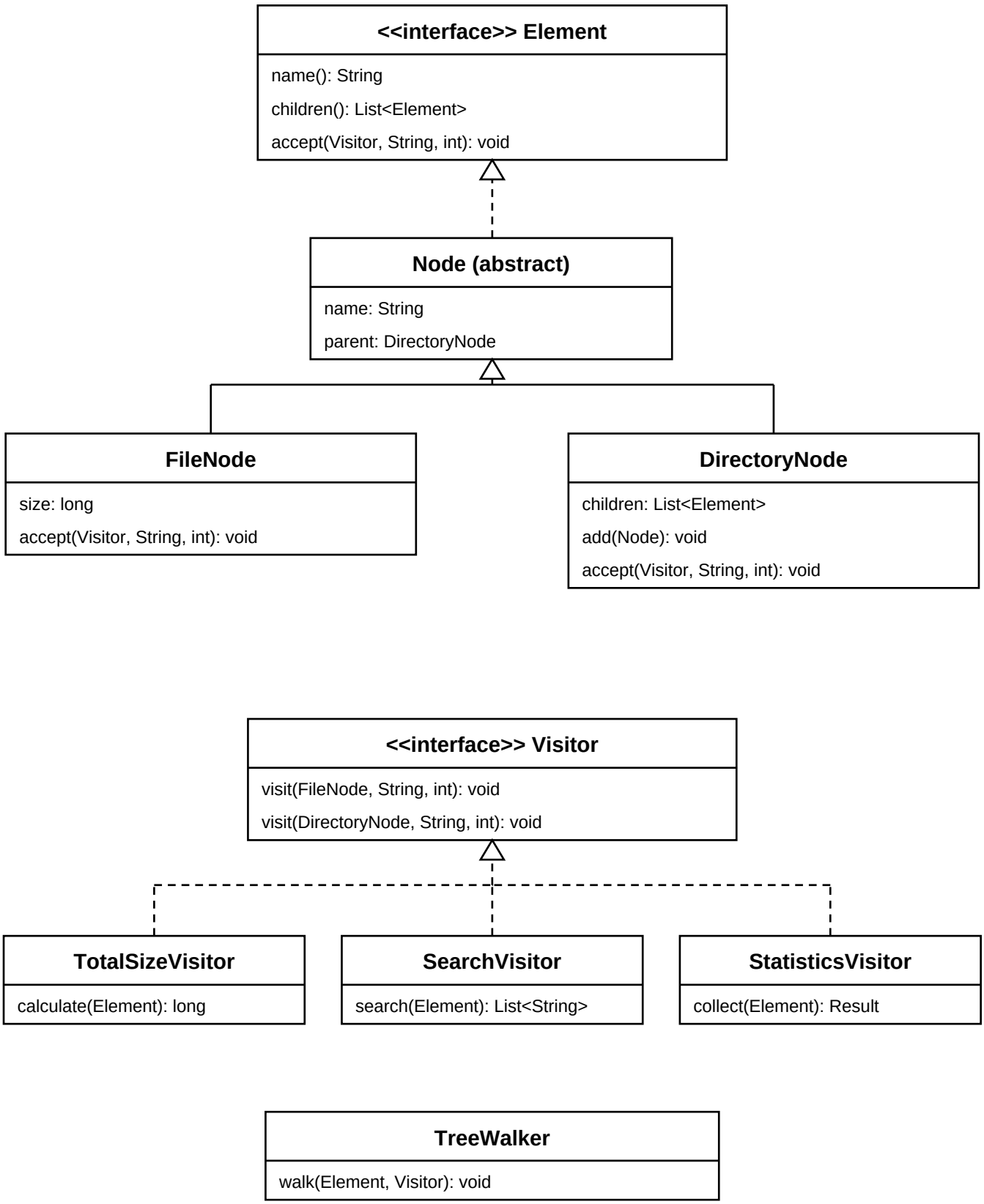
Sample output

```
Total bytes: 740
Matches: [archive/notes-old.txt, lab/notes.txt]
Result[files=4, directories=4, maxDepth=2]
```

Verification and repository

Maven test: 8 passed, 0 failures, 0 errors, 0 skipped. Compiled for Java 17.
FileSystemTest covers both visit overloads, total size, combined search filters, sorted paths, counts and depth, empty and no-match cases, repeat runs, and invalid tree changes.
GitHub: https://github.com/Jasulankoishy/SDP-Ass-6

UML class diagram



DirectoryNode contains zero or more Elements. TreeWalker uses Element and Visitor.

The three concrete visitors use TreeWalker to run their operations.